Command

Command · Undoable editor keystrokes · Build with Waqar

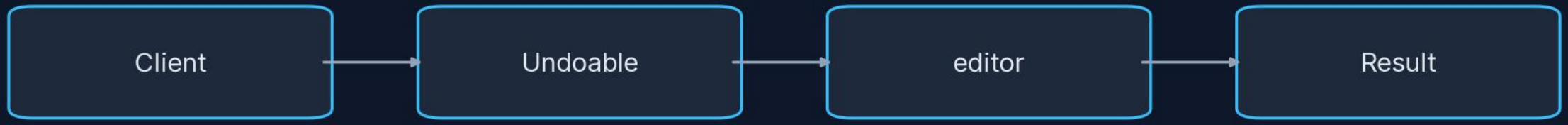
Today's Hook

- Problem: Undoable editor keystrokes
- Pattern: Command — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Undoable editor keystrokes
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Command: Undoable editor keystrokes

Real-World Scenario

- Running example: Undoable editor keystrokes
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.command.problem;

import java.util.*;

public final class NaiveEditor {
  private String text="";
  public void write(String s){ text+=s; }
  public void undo(){ /* cannot undo */ }
  public String text(){ return text; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.command.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;

final class Editor { private final StringBuilder sb=new StringBuilder();
  void write(String s){ sb.append(s); } void deleteLast(int n){ sb.delete(Math.max(0,sb.length()-n), sb.length()); }
  String text(){ return sb.toString(); }
}
interface Command { void execute(); void undo(); }
final class WriteCommand implements Command {
  private final Editor ed; private final String chunk;
  WriteCommand(Editor ed, String chunk){ this.ed=ed; this.chunk=chunk; }
  public void execute(){ ed.write(chunk); } public void undo(){ ed.deleteLast(chunk.length()); }
}
final class CommandBus {
  private final Deque<Command> hist=new ArrayDeque<>();
  void run(Command c){ c.execute(); hist.push(c); }
  void undo(){ if(!hist.isEmpty()) hist.pop().undo(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.command.problem;

import java.util.*;
import java.util.concurrent.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Command: undoable editor ===");
    System.out.println("--- naive ---");
var n=new NaiveEditor(); n.write("Hi"); n.undo(); System.out.println(n.text());
System.out.println("--- command ---");
var ed=new Editor(); var bus=new CommandBus();
bus.run(new WriteCommand(ed,"Hello")); bus.run(new WriteCommand(ed,"!"));
System.out.println(ed.text()); bus.undo(); System.out.println("afterUndo="+ed.text());
  }
}
```

Walkthrough Output

```
=== Command: undoable editor ===
--- naive ---
Hi
--- command ---
Hello!
afterUndo=Hello
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Command.

Related Patterns

- Contrast Command with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Command → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.